LESSON PLAN · PYTHON (GR 6–8) INTERMEDIATE: PYTHON GAME CODERS · LEVEL 2

# Lesson 4: Project: Turtle Art Studio

Module 1 | 60 minutes | functions, input, menus | Grades 7–11 · ages 12–16

**Objective:** Build an interactive drawing program where the user chooses shapes, colours and sizes, with a separate function for each shape.

**Success criteria**

- I can write a separate function for each shape with a size parameter.
- I can build a menu with input and if / elif / else.
- I can keep a program running with a while loop until the user types done.
- I can test each function on its own before joining them together.

## Key vocabulary

- **menu**: A program part that asks the user to choose and then acts on the choice.
- **interactive**: A program that responds to what the user types.
- **loop flag**: A True/False variable, like drawing, that keeps a while loop going.
- **testing**: Running a part of your program to check it works before moving on.
- **.strip().lower()**: Removes extra spaces and makes text small letters.
- **penup / goto**: Lift the pen and jump to a point without drawing.

## You will need

- Projector with Kitabash (lesson video, Python Lab)
- Devices: 1 per learner, with a keyboard
- Printed worksheet (optional)
- Printed project brief
- Project files: art_studio_menu.py

## Timed lesson plan (60 min)

| Time | Stage | What happens |
|---|---|---|
| 5 min | **Warm-up game: Studio Order Desk** | Play Studio Order Desk (unplugged): a menu learner sends orders to shape chefs. Discuss the menu's job and the chefs' jobs. |
| 5 min | **Hook, video and plan** | Show the hook and play the project video. Thumbs poll: "Who already knows which 3 shapes they want?" Learners sketch their studio on paper. |
| 8 min | **Live demo and build-along** | Run the live demo, then everyone builds two shapes and a menu. Test with SQUARE, triangle and banana. |
| 5 min | **Challenge, predict and order** | Shape picker challenge, predict-the-output and the ordering activity. Quick competition: first correct order wins a point for their team. |

| | | |
|---|---|---|
| 18 min | **Build the Art Studio** | Learners build the full studio in breakout rooms. Halfway check-in: every learner has 2 shapes and the done loop working. Teacher visits each room. |
| 6 min | **Debug race** | Race to fix “the shy square”. The winner explains both bugs. |
| 9 min | **Gallery walk and show-and-tell** | Pairs swap and draw a picture with each other’s studio, giving two stars and a wish. Back together, three learners show their artwork live. |
| 4 min | **Submit and exit ticket** | Learners submit the project with a reflection. Exit ticket: the STAR question. |

## Unplugged warm-up

Studio Order Desk (5 min game): one learner is the “menu”, three learners are “shape chefs” who each can draw only one shape on a whiteboard (square, star, triangle). Classmates order “blue star, size big!” The menu sends the order to the right chef; unknown orders get “Sorry, not on the menu”. The order “done” closes the desk. Ask: what was the menu’s job, and what were the chefs’ jobs?

## Teaching the idea

Today learners combine everything from Module 1 into one interactive program. Each shape is its own function with a size parameter, so it can be tested alone. The main program is a menu: a while loop asks for a shape, cleans it with .strip().lower(), and uses if / elif / else to call the right function. done ends the loop. A move_to_spot function keeps shapes apart. Encourage learners to build and test one shape function before writing the menu.

**Analogy:** The studio is like a restaurant: the menu (the while loop with input) takes the order, and each shape function is a chef who only knows one dish. The waiter never cooks; it just sends the order to the right chef.

**Live demo:** 1) Write draw_square(size) and test it with draw_square(80). 2) Write draw_star(size) and test it; ask why 144 degrees works (the star goes round twice: 5 × 144 = 720). 3) Add shape = input("Shape? ") and if/elif/else; type Star and show it fails; add .strip().lower(). 4) Type the size without int() and show the TypeError, then fix it. 5) Wrap the menu in while drawing: and show done ending it. 6) Add move_to_spot(count) and show shapes no longer overlap.

## Questions to ask

| Ask | Listen for |
|---|---|
| Why give each shape its own function? | Each one is short, easy to test and can be reused. |
| Why use .strip().lower() on the shape? | So Star, STAR and " star " all match "star". |
| What stops the while loop? | Typing done sets drawing to False. |
| Why must the size go through int()? | input() gives text and forward() needs a number. |

| | |
|---|---|
| Why define the functions before the loop? | Python must know a function before it is called. |

## Differentiation

**Support**
Give the build-along file and let learners add one shape (star) plus the loop. Two shapes and done is a fine first goal.

**Stretch**
Add a “random” option that picks a random shape, colour and size with random.choice and randint, or a “clear” command that calls t.clear().

## Common misconceptions

| Misconception | How to fix it |
|---|---|
| input() gives a number if you type digits | It always gives text; use int(). |
| "Square" == "square" is True | Capitals matter; use .lower(). |
| Putting drawing = True inside the loop is fine | It must be set once before the loop, or done can never stop it. |
| A function draws when it is defined | It only draws when it is called. |

## Assessment

Project rubric (also on the project brief):

| Criterion | 1 · Beginning | 2 · Developing | 3 · Secure | 4 · Excellent |
|---|---|---|---|---|
| **Program works** | Crashes or draws nothing | Draws one shape, then stops or crashes | Draws every shape the user asks for and stops on done | Works perfectly and handles odd answers like STAR or banana |
| **Functions** | No functions | One function, or functions without parameters | 3 or more shape functions with a size parameter | Extra helper functions (like move_to_spot) and no repeated code |
| **User experience** | No instructions for the user | Asks questions but gives no feedback | Clear menu, feedback after each shape and a final message | Fun title screen, shapes laid out neatly and a celebration at the end |
| **Testing and code quality** | Hard to read; never tested | Some comments; tested once | Each function tested; comments explain each part | Tested by another person and improved from their feedback |

**Exit ticket**

Your menu checks if shape == "star": but the user typed STAR. What one change fixes it?

**Homework**

Let two people at home use your studio. Ask each one to make a picture and screenshot it. Add the shape they asked for that you did not have.

## Answer key — Kitabash activities

### 1. Python Lab: shape picker (perform · python lab)

**Goals checked:** Typing triangle draws a triangle · Calls draw_triangle · Unknown shapes get a Sorry · Uses if, elif and else

**One solution:**

```
import turtle
t = turtle.Turtle()

def draw_square(size):
    for i in range(4):
        t.forward(size)
        t.right(90)

def draw_triangle(size):
    for i in range(3):
        t.forward(size)
        t.left(120)

shape = input("square or triangle? ").strip().lower()
size = int(input("Size? "))
if shape == "square":
    draw_square(size)
elif shape == "triangle":
    draw_triangle(size)
else:
    print("Sorry, I can only draw a square or a triangle.")
```

### 2. Predict the output: growing shapes (perform · code output)

### 3. Plan the studio (perform · ordering)

**Order:** import turtle and make the turtle t → Define a function for each shape → Start a while loop → Ask for a shape (done ends the loop) → Ask for the colour and size → Move to a new spot and call the right shape function

### 4. Project: Turtle Art Studio (perform · python lab)

**Goals checked:** Runs a full session · Draws in 3 colours · Counts the shapes · Handles unknown shapes · Uses functions, a loop and a menu

**Example program:**

```
# Turtle Art Studio
# Made by: Arjun
import turtle
t = turtle.Turtle()
t.speed(0)

def draw_square(size):
    for i in range(4):
        t.forward(size)
        t.right(90)

def draw_triangle(size):
    for i in range(3):
        t.forward(size)
        t.left(120)

def draw_star(size):
    for i in range(5):
        t.forward(size)
        t.right(144)

def draw_flower(size):
    for i in range(8):
        t.circle(size / 2)
        t.right(45)

def move_to_spot(number):
    t.penup()
    t.goto(-180 + number * 120, 0)
    t.pendown()

print("=== TURTLE ART STUDIO ===")
print("Shapes: square, triangle, star, flower. Type done to finish.")
print("Colours: red, orange, yellow, green, blue, purple, pink, gold...")
count = 0
drawing = True
while drawing:
    shape = input("Shape? ").strip().lower()
    if shape == "done":
        drawing = False
    elif shape in ["square", "triangle", "star", "flower"]:
        colour = input("Colour? ").strip().lower()
        size = int(input("Size (20 to 100)? "))
        move_to_spot(count)
        t.color(colour)
        if shape == "square":
            draw_square(size)
        elif shape == "triangle":
            draw_triangle(size)
        elif shape == "star":
            draw_star(size)
```

```
        else:
            draw_flower(size)
        count += 1
        print(f"Drew a {colour} {shape}. Shapes so far: {count}")
    else:
        print("Sorry, I don't know that shape.")

print(f"Your artwork has {count} shapes. Beautiful!")
t.hideturtle()
```

**5. Debug it: the shy square** (probe · python lab)

**Goals checked:** Square draws a square · Runs without errors

**One solution:**

```
import turtle
t = turtle.Turtle()

def draw_square(size):
    for i in range(4):
        t.forward(size)
        t.right(90)

shape = input("Shape? ").strip().lower()
size = int(input("Size? "))
if shape == "square":
    draw_square(size)
```

## Answer key — worksheet

1. **Key words:** menu = A program part that asks the user to choose and then acts on the choice. · interactive = A program that responds to what the user types. · loop flag = A True/False variable, like drawing, that keeps a while loop going. · testing = Running a part of your program to check it works before moving on.
2. **Plan your program:** Goals: Typing triangle draws a triangle; Calls draw_triangle; Unknown shapes get a Sorry; Uses if, elif and else. One solution: import turtle t = turtle.Turtle() def draw_square(size): for i in range(4): t.forward(size) t.right(90) def draw_triangle(size): for i in range(3): t.forward(size) t.left(120) shape = input("square or triangle? ").strip().lower() size = int(input("Size? ")) if shape == "square": draw_square(size) elif shape == "triangle": draw_triangle(size) else: print("Sorry, I can only draw a square or a triangle.")
3. **Predict the output: growing shapes:** A. 100
4. **Plan the studio:** 1 import turtle and make the turtle t · 2 Define a function for each shape · 3 Start a while loop · 4 Ask for a shape (done ends the loop) · 5 Ask for the colour and size · 6 Move to a new spot and call the right shape function
5. **Plan your program:** Goals: Square draws a square; Runs without errors. One solution: import turtle t = turtle.Turtle() def draw_square(size): for i in range(4): t.forward(size) t.right(90) shape = input("Shape? ").strip().lower() size = int(input("Size? ")) if shape == "square": draw_square(size)

6. **My plan:** A clear plan that covers every requirement.
7. **Checklist:** All requirements met.
8. **Two stars and a wish:** Kind, specific feedback and an improvement.

Kitabash original curriculum.